

Take-Home Assignment

Software Engineer — Flutter / Next.js

“Session Replay” — a communication history & analytics feature for an AAC app

1. Context

Avaz builds AAC (Augmentative and Alternative Communication) apps that give non-verbal or minimally-verbal people — a way to communicate by tapping symbols that speak on their behalf.

Every screen in our product effectively serves two very different people at once:

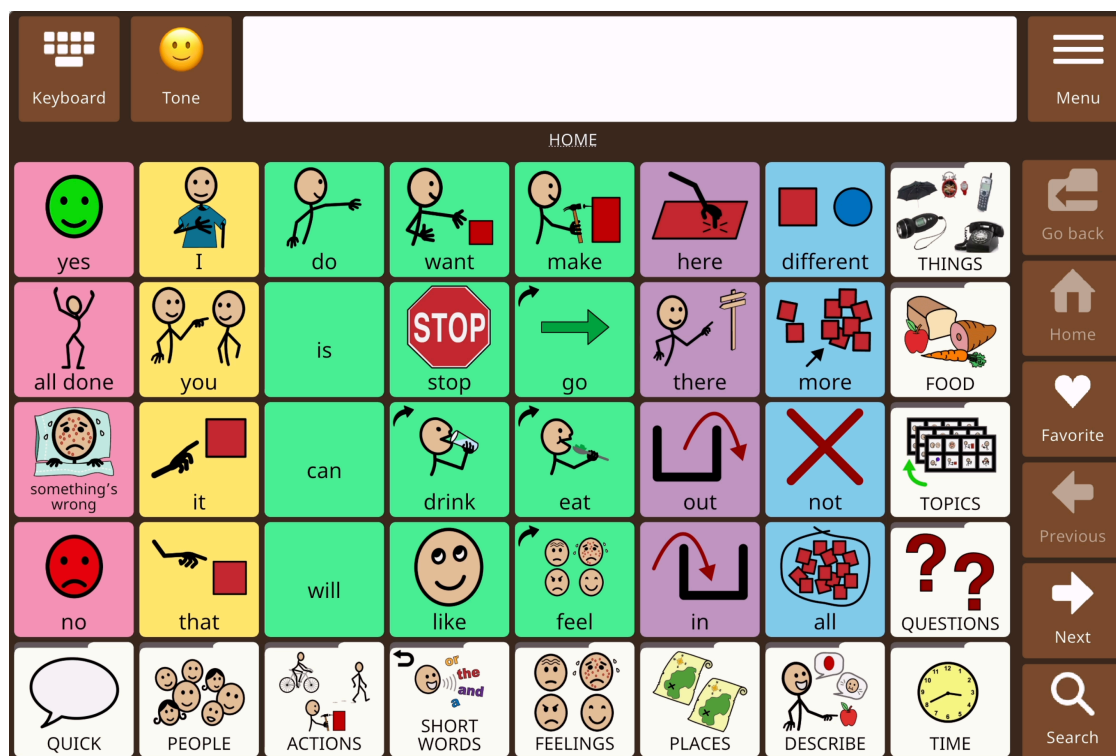
- The caregiver / therapist / parent — who sets up content the end-user will use.
- The end-user — the communicator. They may have limited fine motor control, limited reading ability.

This dual-persona reality shapes almost every product and engineering decision we make — including the ones we'd like you to make in this exercise.

2. The problem

Design and build a small but real feature: **Session Replay** — every tile the end-user taps during normal use is quietly captured as a communication session, and a caregiver can later browse past sessions and “replay” one: see the sequence of phrases spoken, in order, and play them back.

For reference, this is the existing AvazAAC end-user board — a grid of large, tappable symbol tiles with a sentence bar at top. Build a tap surface like this one as the screen every Session Replay event is captured from:



Existing AvazAAC home board — end-user tap surface. Reference only: you do not need to replicate the full folder navigation (TOPICS, PEOPLE, ACTIONS, etc.), the exact symbol set, or the color-coding scheme — a flat grid of

large, tappable phrase tiles with a sentence bar is enough. What matters is that tapping a tile speaks it with immediate feedback, exactly like this board does.

Concretely, the system needs two modes / views:

End-user mode — a tap surface like the reference board above: a grid of large, tappable tiles with a sentence bar. They keep tapping tiles to speak. Behind the scenes, each tap is captured as an event (what was spoken, when) into a local, append-only session log. Capturing this log must add no perceptible delay or jank to the tap-to-speak feedback loop — logging is a side effect the end-user should never notice.

Caregiver mode — a list of past sessions (e.g. grouped by day / device), and a replay view for a selected session: a readable transcript of the phrases spoken, in order, with the ability to play the session back (step through it, or auto-play it) and hear each phrase again, with some visible sense of timing or pacing between taps.

The end-user device is frequently used in places with unreliable or no connectivity — a classroom, a therapy clinic, a car, an outdoor session — so session capture must keep working fully offline, and the caregiver must be able to browse and replay previously-synced sessions offline too. Sessions captured on one device (or a companion caregiver device) need to reach the backend, and reach other devices tied to the same end-user, without silently losing or duplicating events.

3. What we want you to build

Pick ONE frontend track based on your background:

- Flutter mobile app
- Next.js web app

Either way, back it with a backend API service that owns persistence and exposes the sync/API surface. This mirrors how our stack actually splits: a client that must work offline-first, and a backend of record.

Minimum functional scope:

- Implement an end-user tap surface like the reference board shown in Section 2 — a grid of large, tappable phrase tiles with a sentence bar. You do not need to rebuild caregiver content-editing, the full symbol library, or folder navigation (TOPICS, PEOPLE, etc.) — a flat grid of phrase tiles is enough. Every tap is captured locally as a timestamped event, with no added latency to the existing tap → speak feedback loop.
- Taps are grouped into sessions using some segmentation rule of your choosing (e.g. an idle-time cutoff, explicit start/stop, app foreground/background) — state and justify the rule you picked.
- Caregiver mode lists past sessions and can open one to see a transcript and replay it — spoken, played — in order.
- Session data persists locally across app restarts, and both capturing new sessions and browsing/replaying past ones are fully usable with no network connection.
- A sync path exists between the local store and the API backend, in both directions, that a reviewer can actually exercise (e.g., capture a session offline on one device, go back online, show it lands on the backend and becomes visible from a second device).

4. Deliberately left open

- Exactly what counts as a loggable event — only a completed tile tap, or also things like navigating into a folder/category, clearing the sentence bar, or switching keyboards/tones. Define the event schema and say what you included and why.
- Where one session ends and the next begins (idle-time cutoff, explicit boundary, calendar-day boundary, something else).

- Replay fidelity — whether playback follows the original inter-tap timing, a normalized/compressed pacing. We do not expect a production-grade player; we do expect you to name the trade-off and pick something defensible.
- How “speaking” a phrase during replay is actually implemented — real device TTS or a pre-recorded clip — same as the underlying tile-tap playback, or different if you have a reason.
- The exact reconciliation rule when sessions/events from two devices (e.g. the end-user's tablet and a companion caregiver device, or the same tablet after a factory-reset-and-restore) reach the backend out of order, overlapping, or with clock drift between them.
- How closely to match the reference board's visual design — a flat grid of large, tappable tiles with a sentence bar satisfies the requirement; replicate more of the folder navigation or color-coding only if you want to.
- Any notion of “users” or “accounts”, or of multiple end-user profiles sharing one backend — a single hardcoded end-user/board is fine unless you decide otherwise (say why).
- How long sessions are retained, and whether/how a caregiver can delete one — a session transcript is a real communication record, so this isn't purely a technical choice.

If you find yourself unsure whether something is in scope, make the call, write it down, and move on. Do not come back to us with clarifying questions — resolving this kind of ambiguity independently is exactly what we're testing.

5. What to submit

1. Email: Email to abhimanyu@avaz.in , balajik@avaz.in with

subject: “<Your Name> Invention Labs - SDE II - Full Stack (Flutter & Web) 2026”

2. Working code — a repo (a link to a private repo you invite us to) containing the frontend app and the backend API service, plus setup instructions so we can run it locally in under 10 minutes.

Add the following GitHub users as collaborators:

- [abhimanyuavaz](#)
- [ilabsbalaji](#)

3. README.md (this is the part we weigh most heavily) — This should have three sections:

Section 1:

- How to run it, and any assumptions needed just to get it started (dependencies, seed data, etc.).

Section 2:

- The assumptions you made for each open question in 4, and why.
- Your architecture: how the client, local event log, and backend are structured, and why you split it that way.
- Your data model and your sync/conflict-handling approach, including what happens when two devices produce overlapping or out-of-order events before reconnecting.
- At least one alternative approach you considered and rejected for a non-trivial decision, and why.
- End-user experience decisions you made specifically to guarantee that capturing and syncing session data never adds latency, visible interruption, or a “please wait” moment to the person mid-communication.

Section 3:

- What you would do differently, or add next, with another 2 days.

4. (Optional) A 2–3 minute screen recording showing: normal end-user tapping generating a session, the caregiver browsing and replaying that session, and a go-offline / capture a session / reconnect sequence that reaches a second device or browser. Not required, but it saves everyone time.

6. What we validate in your submission

We're looking at how you translate an ambiguous, real-world problem into a working system — not just whether the code runs. Specifically, we look at:

- Architecture & data model — is there a clear separation between the app, the local event log, and the backend; would the data model still hold up if we added a few more analytics-style features on top of session history next quarter, or would it need a rewrite.
- Offline-first & sync design — does session capture keep working with zero network connectivity and zero added latency; when a device reconnects, is there a stated, reasoned rule for reconciling overlapping or out-of-order events that you can explain and demonstrate — not one that silently drops or duplicates data. We will check the sync behaviour by installing the app on multiple devices/browsers.
- API design — do the API endpoints map to real resources (sessions, events) with sensible status codes and error handling, and are they actually shaped for this sync use case.
- End-user experience — concrete choices you made specifically so that background logging and syncing never introduce delay, jank, or an interruption for someone mid-communication who cannot “wait and retry.”
- Handling ambiguity — how you resolved the open questions in 4, whether your assumptions are written down rather than silently baked in.
- Code quality & testing — whether the code is organized in a way a teammate could pick up, and whether your testing effort (if any) went toward the riskiest logic — event ordering, session segmentation, and sync/merge — rather than only the easy paths.
- Communication — whether README.md reads like a genuine account of your thinking: what you tried, what you rejected and why, what you'd change with more time, and an honest, specific description of how you used AI tools.